

Module 61CSE215: Object-oriented Programming with Java

Basic Input Output, Variables, Data Types, Operators

Vietnamese-German University

Ngoc Tran, Ph.D.

ngoc.th@vgu.edu.vn

Binh Duong, 2025

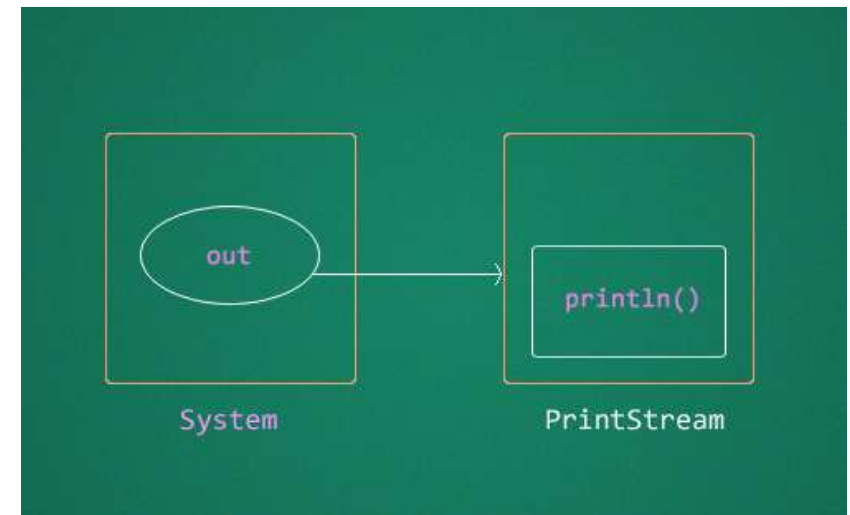
Content

- Basic Input and Output
- Data Types
- Variables
- Operators

Basic Output

A final class is a **class that can't be extended**. Also methods could be declared as final to indicate that cannot be overridden by subclasses.

- **System** is a **final class** in **java.lang** package.
 - **out** is a **static member field** of **System** class and is of type **PrintStream**.
 - **Print** & **println** are methods of **PrintStream** class.
-
- `System.out.println(data);` prints data at the console and moves the cursor to the start of the next line.
 - `System.out.print(data);` prints data on the console and the cursor remains at the end of data.
 - **Data** can be string, float, byte, double, integer, ...



- In Java, **static fields belongs to the class, not instances of the class**. Thus, **all instances of any class will access the same static field variable**.
- A non-static field value can be different for every object (instance) of a class.

Example 1

```
int i = 100;
char ch = 'C';
double d = 2.5;
float f = 4.9f;
boolean bool = true;
String s = "Hello Java";

System.out.println();
System.out.println(i);
System.out.println(ch);
System.out.println(d);
System.out.println(f);
System.out.println(bool);
System.out.println(s);
System.out.println(" OOP!");
```

Example 2

```
System.out.println("Using print()");
```

```
System.out.print("1 ");
```

```
System.out.print("2 ");
```

```
System.out.print("3");
```

```
System.out.println();
```

```
System.out.println("...\n");
```

```
System.out.println();
```

```
System.out.println("n-2");
```

```
System.out.println("n-1");
```

```
System.out.println("n");
```

Exercises

1. Run the following Java codes and write down the results.

Code 1:

```
int num1 = 10, num2 = 20, sum;  
  
System.out.print("The addition of ");  
System.out.println(num1 + " and " + num2 + " is:");  
System.out.println(num1 + num2);
```

Code 2:

```
System.out.print(num1 + " + " + num2 + " = ");  
System.out.println(num1 + num2);
```

Code 3:

```
System.out.println(num1 + " + " + num2 + " = " + num1 + num2);
```

Exercises

2. Given $n = 10$. Using the basic output function to display the empty right triangle (see Figure a) and the empty isosceles triangle (see Figure b).



a)



b)

Java is Case-Sensitive

- E.g., **Main** is different from **main**, 'A' is different from 'a'.
- The name of the java file **must match** the class name. When saving the file, save it using the class name and add ".java" to the end of the filename.

```
public class Main {  
    public static void main(String[] args) {  
        System.out.println("Hello World");  
    }  
}
```

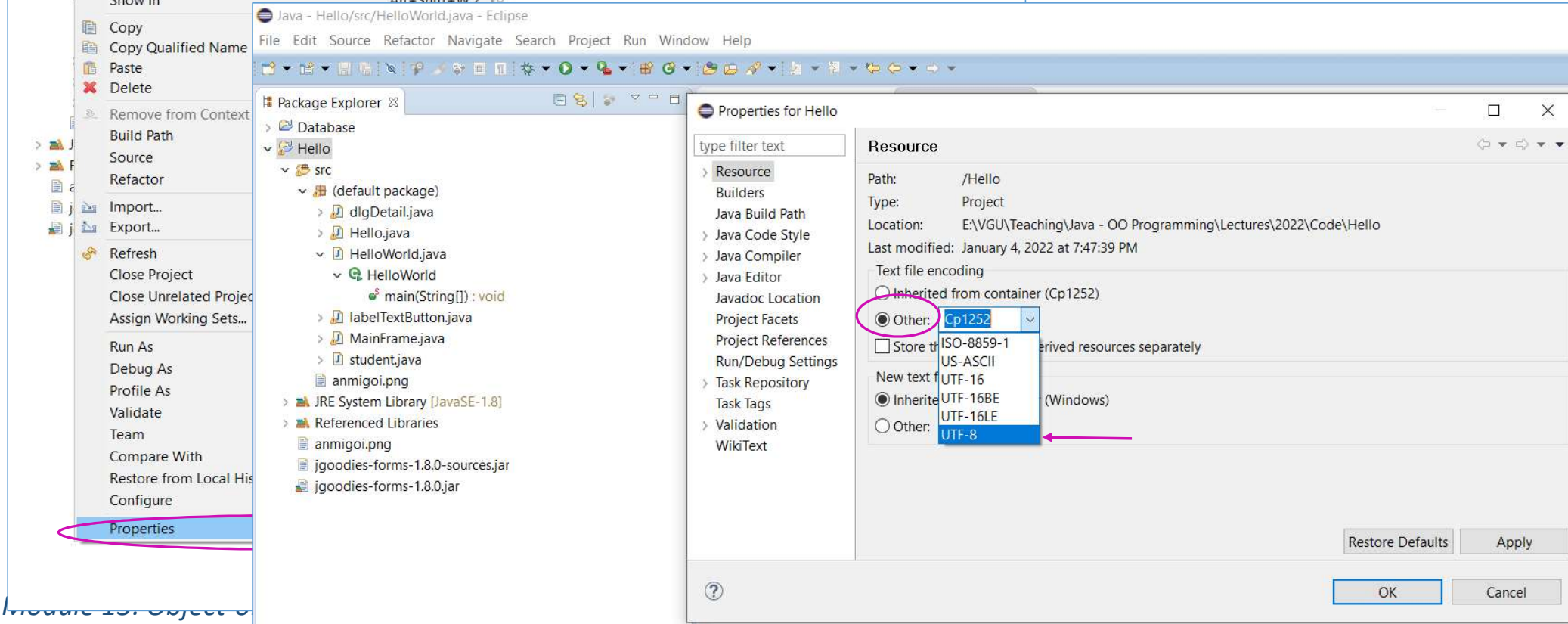
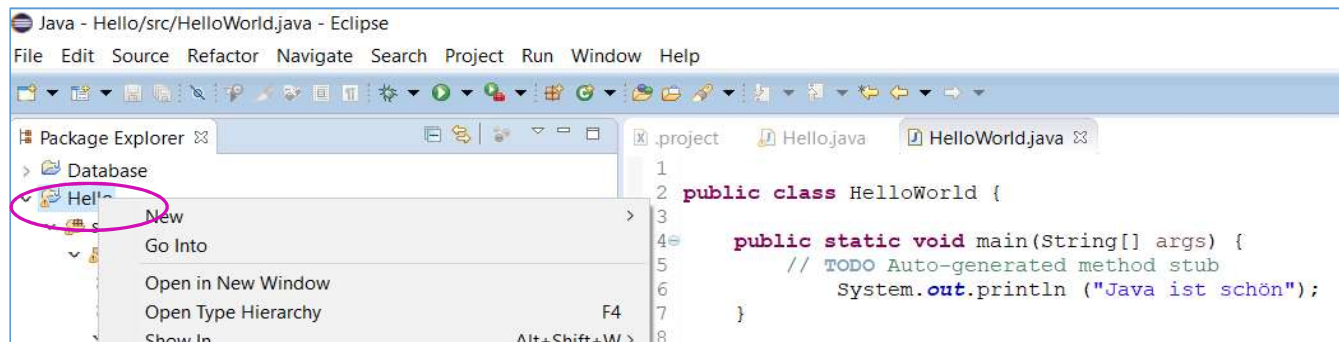

Identifier

- A Java **identifier** is a name given to a **package**, **class**, **interface**, **method**, or **variable**.
- It allows a programmer to refer to the item from other places in the program.

```
34 // Class Declaration
35 public class Dog {
36     // Instance Variables
37     String breed;
38     String size;
39     int age;
40     String color;
41
42
43     // method 1
44     public String getInfo() {
45         return ("Breed is: "+breed+" Size is:"+size+" Age is:"+age+" color is: "+color);
46     }
```

Identifier

- Java identifiers include **letters (a-z)**, **digits (0-9)**, **underscore (_)**, **dollar sign (\$)**.
- Java letters and digits can be anything from the Unicode character set, which means characters in Chinese, Japanese, and other languages can be used.
 - How to configure the Unicode character for use? (see next page).
- **Spaces** are **not acceptable**.
- The **length** does **not matter**, so you can have a really long identifier if you choose.
- A compile-time error will occur if the identifier uses the same spelling as a **keyword**, the **null** literal, or **boolean** literal.



Identifier Samples

- Consider these identifiers. Are they valid or invalid?
 - `_variablename`
 - `_3variable`
 - `$testvariable`
 - `VariableTest`
 - `variabletest`
 - `this_is_a_variable_name_that_is_long_but_still_valid_because_of_the_underscores`
 - `max_value`
- They are **valid**

Identifier Samples

- Consider these identifiers. Are they valid or invalid?
 - `8example`
 - `exa+ple`
 - `variable test`
 - `this_long_variable_name_is_not_valid_because_of_this-hyphen`
- They are **invalid**

Variables

- Variables are containers which holds the value while the Java program is executed.
- In Java, all variables must be declared before they can be used.
- Syntax of declaring a variable:

data_type identifier [= value][, identifier [= value] ...];

- [] means optional.
- *Identifier* is the name of the variable as ruled in slide 11.
- *Value* is initializer.
- *Datatype* (see slide 15)

```
int a,b,c;  
float pi;  
double d;  
char a;
```

```
int i = 0;  
int j = 1, k = 0;
```

Data Types

- Data types are divided into two groups:
 - **Primitive** data types
 - byte, short, int, long, float, double, boolean and char.
 - **Non-primitive** data types:
 - String, Arrays, Classes, ...

Primitive Data Type



Data Type	Size	Description
byte	1 byte	Stores whole numbers from -128 to 127
short	2 bytes	Stores whole numbers from -32,768 to 32,767
int	4 bytes	Stores whole numbers from -2,147,483,648 to 2,147,483,647
long	8 bytes	Stores whole numbers from -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
float	4 bytes	Stores fractional numbers. Sufficient for storing 6 to 7 decimal digits
double	8 bytes	Stores fractional numbers. Sufficient for storing 15 decimal digits
boolean	1 bit	Stores true or false values
char	2 bytes	Stores a single character/letter or ASCII values

Example

```
int myNum = 5;           // Integer (whole number)
float myFloatNum = 5.99f; // Floating point number
char myLetter = 'D';     // Character
boolean myBool = true;   // Boolean
String myText = "Hello"; // String
```

Data Type

- Java is a **strongly typed language**.
- Java's **safety** and **robustness** comes from this fact.
- Every variable has a type, every expression has a type, and every type is strictly defined.
- All assignments, whether explicit or via parameter passing in method calls, are checked for **type compatibility** by compiler.
- There are no automatic coercions or conversions of conflicting types as in some languages.

Character Literals

- All of the visible ASCII characters can be directly entered inside the **quotes**, such as 'a', 'z', and '@'.
- For characters that are impossible to enter directly, using the **escape sequences** that the string starts with the **backslash ' \ '**.

Escape sequence	Description	Escape sequence	Description
\ddd	Octal Characters, e.g., \141 is 'a'	\r	Carriage Return
\uxxxx	Hexadecimal Unicode Characters, e.g., \u0061 is ISO Latin 1 'a'	\n	New Line (or, Line Feed)
\'	Single quote '	\f	Form Feed
\"	Double quotes "	\t	Tab
\\	Backslash	\b	Backspace

Character Literals

```
System.out.println("This is before\nNow new line");  
System.out.println("TEXTBEFORE\rOverlap");  
System.out.println("12\b3");
```

Output:

```
This is before  
          Now new line  
OverlapORE  
13
```

Exercise

- Display the below string with the pair of double quotes at the console.

“Hello the world!”

Type Conversion & Casting

- When one type of data is assigned to another type of variable, an **automatic type conversion** will take place if the following two conditions are met:
 1. The two types are compatible.
 2. The destination type is larger than the source type.

Type Conversion & Casting

- Example: the `int` type is always large enough to hold all valid `byte` values, so no explicit cast statement is required

```
int i;  
byte b = 21;  
i = b;
```

```
long l;  
i = 54678;  
l = i;
```

Type Conversion & Casting

- To create a conversion between two **incompatible** types, you must use a **cast**.
- A **cast** is simply an **explicit** type conversion. **Cast** is sometimes called a narrowing conversion, since you are explicitly making the value narrower so that it will fit into the target type.
- Cast Syntax:

`(target-type) value`

where **target-type** specifies the desired type to convert the specified value to.

- *Example*: the following fragment casts an int to a byte.

```
int a;  
byte b;  
// ...  
b = (byte) a;
```


Exercises

- Run the following code and write down the results.

```
public class Conversion {  
  
    public static void main(String[] args) {  
        byte b;  
        int i = 257;  
        double d = 323.142;  
  
        System.out.println("\nConversion of int to byte.");  
        b = (byte) i;  
        System.out.println("i and b " + i + " " + b);  
  
        System.out.println("\nConversion of double to int.");  
        i = (int) d;  
        System.out.println("d and i " + d + " " + i);  
  
        System.out.println("\nConversion of double to byte.");  
        b = (byte) d;  
        System.out.println("d and b " + d + " " + b);  
    }  
}
```

Exercises

- Run the following code and write down the results.

```
public class Conversion {

    public static void main(String[] args) {
        byte b;
        int i = 257;
        double d = 323.142;

        System.out.println("\nConversion of int to byte.
        b = (byte) i;
        System.out.println("i and b " + i + " " + b);

        System.out.println("\nConversion of double to int.
        i = (int) d;
        System.out.println("d and i " + d + " " + i);

        System.out.println("\nConversion of double to byte.
        b = (byte) d;
        System.out.println("d and b " + d + " " + b);
    }
}
```

This program generates the following output:

Conversion of int to byte.
i and b 257 1

Conversion of double to int.
d and i 323.142 323

Conversion of double to byte.
d and b 323.142 67

Basic Input

- `Scanner` allows the user to take input from the console.
- It is in the `java.util` package.
- Syntax to declare a scanner object:

```
Scanner sc = new Scanner(System.in);
```

- `System.in` is a standard input stream.

Basic Input

Method	Description
int nextInt()	Input is an integer.
float nextFloat()	Input is a float.
double nextDouble()	Input is a double.
byte nextByte()	Input is a byte.
String nextLine()	Input is a string.
boolean nextBoolean()	Input is a Boolean.
long nextLong()	Input is a Long.
short nextShort()	Input is a Short.
BigInteger nextBigInteger()	Input is a BigInteger.
BigDecimal nextBigDecimal()	Input is a BigDecimal.

Example – Input numbers

```
import java.util.Scanner;
class BasicInput1 {
    public static void main(String args[])
    {
        Scanner sc = new Scanner(System.in);
        String s = sc.nextLine();
        System.out.println("You entered string " + s);
        int i = sc.nextInt();
        System.out.println("You entered integer " + i);
        float f = sc.nextFloat();
        System.out.println("You entered float " + f);
        sc.close();
    }
}
```

Example – Input string

```
import java.util.*;
class BasicInput2
{
    public static void main(String[] args)
    {
        Scanner sc= new Scanner(System.in);
        System.out.print("Enter a string: ");
        String s = sc.nextLine();
        System.out.print("You have entered: " + s);
        sc.close();
    }
}
```

Operators

- Arithmetic Operators
- Relational Operators
- Boolean Logical Operators
- Assignment Operator
- ? Operator
- Operator Precedence

Arithmetic Operators

Operator	Result
+	Addition (also unary plus)
-	Subtraction (also unary minus)
*	Multiplication
/	Division
%	Modulus
++	Increment
+=	Addition assignment
-=	Subtraction assignment
*=	Multiplication assignment
/=	Division assignment
%=	Modulus assignment
--	Decrement

```
int a = 5; // a is 5
```

```
int b = -a; // b is -5
```

```
int c = -b; // c is +5
```

Compound
assignment
operators

Arithmetic Operators

```
public class BasicMath {  
  
    public static void main(String[] args) {  
        System.out.println("Integer  
Arithmetic");  
        int a = 1 + 1;  
        int b = a * 3;  
        int c = b / 4;  
        int d = c - a;  
        int e = -d;  
        System.out.println("a = " + a);  
        System.out.println("b = " + b);  
        System.out.println("c = " + c);  
        System.out.println("d = " + d);  
        System.out.println("e = " + e);  
    }  
}
```

```
System.out.println("\nFloating Point  
Arithmetic");  
        double da = 1 + 1;  
        double db = da * 3;  
        double dc = db / 4;  
        double dd = dc - da;  
        double de = -dd;  
        System.out.println("da = " + da);  
        System.out.println("db = " + db);  
        System.out.println("dc = " + dc);  
        System.out.println("dd = " + dd);  
        System.out.println("de = " + de);  
    }  
}
```

Arithmetic Operators

Solution:

Integer Arithmetic

a = 2

b = 6

c = 1

d = -1

e = 1

Floating Point Arithmetic

da = 2.0

db = 6.0

dc = 1.5

dd = -0.5

de = 0.5

Arithmetic Operators

- Compound assignment operators

```
public class OpEquals {  
  
    public static void main(String[] args) {  
        int a = 1;  
        int b = 2;  
        int c = 3;  
  
        a += 5; // a = a + 5  
        b *= 4; // b = b * 4  
        c += a * b; // c = c + a * b  
        c %= 6; // c = c % 6  
  
        System.out.println("a = " + a);  
        System.out.println("b = " + b);  
        System.out.println("c = " + c);  
    }  
}
```

Arithmetic Operators

- Compound assignment operators

```
public class OpEquals {  
  
    public static void main(String[] args) {  
        int a = 1;  
        int b = 2;  
        int c = 3;  
  
        a += 5;  
        b *= 4;  
        c += a * b;  
        c %= 6;  
  
        System.out.println("a = " + a);  
        System.out.println("b = " + b);  
        System.out.println("c = " + c);  
    }  
}
```

Arithmetic Operators

- Modulus Operator

- Given $x = 42$, $y = 42.25$. Program to output the remainder of x and y divided by 10.

```
public class Modulus {  
  
    public static void main(String[] args) {  
        int x = 42;  
        double y = 42.25;  
  
        System.out.println("x mod 10 = " + x % 10);  
        System.out.println("y mod 10 = " + y % 10);  
    }  
}
```

When you run this program, you will get the following output:
x mod 10 = 2
y mod 10 = 2.25

Increment/Decrement Operators

```
public class IncDec {  
  
    public static void main(String[] args) {  
        int a = 1;  
        int b = 2;  
        int c;  
        int d;  
  
        c = ++b;  
        d = a++;  
        c++;  
  
        System.out.println("a = " + a);  
        System.out.println("b = " + b);  
        System.out.println("c = " + c);  
        System.out.println("d = " + d);  
    }  
}
```

The output of this program follows:

```
a = 2  
b = 3  
c = 4  
d = 1
```

```
q = 1  
c = 4  
p = 3  
g = 5
```

The output of this program follows:

Relational Operators

Operator	Result
==	Equal to
!=	Not equal to
>	Greater than
<	Less than
>=	Greater than or equal to
<=	Less than or equal to

- The relational operators determine the relationship that one operand has to the other. Specifically, they determine **equality** and **ordering**.
- The outcome of these operations is a **boolean** value.
- The relational operators are most frequently used in the expressions that control the **if statement** and the various **loop statements**.

Relational Operators

- Example

```
int a = 4;
```

```
int b = 1;
```

```
boolean c = a < b;
```

➔ Output: ?

```
int done;  
//...  
if(!done)... // Valid in C/C++  
if(done)...  // but not in Java.
```

In Java, these statements must be written like this:

```
if(done == 0)... // This is Java-style.  
if(done != 0)...
```


Boolean Logical Operators

- The Boolean logical operators shown here operate only on boolean operands.
- All of the binary logical operators combine two boolean values to form a resultant boolean value

Operator	Result
&	Logical AND
	Logical OR
^	Logical XOR (exclusive OR)
	Short-circuit OR
&&	Short-circuit AND
!	Logical unary NOT
&=	AND assignment
=	OR assignment
^=	XOR assignment
==	Equal to
!=	Not equal to
?:	Ternary if-then-else

Boolean Logical Operators

- The OR operator results in **true** when A is **true**, no matter what B is.
- The AND operator results in **false** when A is **false**, no matter what B is.

A	B	A B	A & B	A ^ B	!A
False	False	False	False	False	True
True	False	True	False	True	False
False	True	True	False	True	True
True	True	True	True	False	False

Boolean Logical Operators

```
public class BoolLogic {

    public static void main(String[] args) {
        boolean a = true;
        boolean b = false;
        boolean c = a | b;
        boolean d = a & b;
        boolean e = a ^ b;
        boolean f = (!a & b) | (a & !b);
        boolean g = !a;

        System.out.println("a = " + a);
        System.out.println("b = " + b);
        System.out.println("a|b = " + c);
        System.out.println("a&b = " + d);
        System.out.println("a^b = " + e);
        System.out.println("!a&b|a&!b = " + f);
        System.out.println("!a = " + g);
    }
}
```

Output:

```
a = true
b = false
a|b = true
a&b = false
a^b = true
!a&b|a&!b = true
!a = false
```

Boolean Logical Operator

- **Short Circuit Logical Operators:** Java will **not** bother to evaluate the right-hand operand when the outcome of the expression can be determined by the left operand alone.

- Example 1:

```
if (denom != 0 && num / denom > 10)
```

Note: If this line of code were written using the single **&** version of AND, both sides would be evaluated.

- Example 2:

```
if(c==1 & e++ < 100) d = 100;
```

Assignment Operators

- Example

```
int x, y, z;  
x = y = z = 100; // set x, y, and z to 100
```

the value of $z = 100$ is 100, which is then assigned to y , which in turn is assigned to x .

- Using a “**chain of assignment**” is an easy way to set a group of variables to a **common value**.

? Operators

- Java includes a special **ternary (three-way) operator** that can replace certain types of if-then-else statements. This operator is the ?
- Syntax: `expression1 ? expression2 : expression3`
 - If **expression1** is **true**, then **expression2** is evaluated; otherwise, **expression3** is evaluated.
 - *The result of the ? operation is that of the expression evaluated.*
 - Both **expression2** and **expression3** are required to return the same (or compatible) type, which **can't be void**.

? Operators

- Example: `ratio = denom == 0 ? 0 : num / denom;`
 - If `denom` equals `zero`, the expression **between the question mark and the colon** is evaluated and used as the value of the entire `?` expression.
 - If `denom` does **not** equal `zero`, the expression **after the colon** is evaluated and used for the value of the entire `?` expression.
 - The result, produced by the `?` Operator, is then assigned to `ratio`.

Exercises

1. Input 3 integers and output their addition at the console.
2. Input a number of days at the console. Given the light speed is 186000 miles/second. Computes and output the number of miles that light will travel in a specified number of days.
3. Input a Celsius degree x at the console. Convert x to the Fahrenheit degree y . Output y at the console.
4. Input a Fahrenheit degree y . Convert x to the Celsius degree x . Output y at the console.
5. Input arguments of a , b , h of a trapezoid where a is the bottom base, b is the top base, h is the height. Output the area of trapezoid.

Hint: $\text{Area} = (\text{sum of parallel sides} \div 2) \times \text{height}$.

Exercises

How to Convert Fahrenheit to Celsius

$$C = \frac{5}{9} (F - 32)$$

$$F = \frac{9}{5} C + 32$$

Example: convert 50 °F to: °C

$$50 \text{ °F} = (50 - 32) \times 5/9 = 10 \text{ °C}$$

Exercises

6. Input the diameter d of a circle at the console. Output the area S and the circumference C of the circle.

Hint: use `Math.PI` for the pi value.

7. Input the 3 edges' size of the triangle x , y , z at the console. Use ? Operator to find out if their triangle type is isosceles triangle or equilateral triangle.

8. Given a square triangle ABC with the right angle $\hat{A}$. Input the 2 sides' length a , b at the console. Calculate the hypotenuse length c using Pythagoras theorem.

Hint: use `Math.sqrt()`

Exercise 1 – Solution

```
import java.util.*;

class Exercisel
{
    public static void main(String[] args)
    {
        Scanner sc = new Scanner(System.in);
        System.out.print("Input number 1: ");
        int n1 = sc.nextInt();
        System.out.print("Input number 2: ");
        int n2 = sc.nextInt();
        System.out.print("Input number 3: ");
        int n3 = sc.nextInt();
        int n = n1 + n2 + n3;
        System.out.println("Total= " + n);
        sc.close();
    }
}
```

Operator Precedence

Highest						
++ (postfix)	-- (postfix)					
++ (prefix)	-- (prefix)	~	!	+ (unary)	- (unary)	(type-cast)
*	/	%				
+	-					
>>	>>>	<<				
>	>=	<	<=	instanceof		
==	!=					
&						
^						
&&						
?:						
->						
=	op=					
Lowest						

e.g., *=, +=,
%=, ^=

- Operators in the same row are equal in precedence.
- In binary operations, the order of evaluation is left to right (except for assignment, which evaluates right to left).
- Although they are technically separators, the [], (), and . can also act like operators. In that capacity, they would have the highest precedence.
- Also, notice the arrow operator (->). It is used in lambda expressions.
- The java "instanceof" operator is used to test whether the object is an instance of the specified type (class or subclass or interface)

Operator Precedence

Example: Operator Precedence

```
class Precedence {  
    public static void main(String[] args) {  
  
        int a = 10, b = 5, c = 1, result;  
        result = a-++c-++b;  
  
        System.out.println(result);  
    }  
}
```

References

1. Herbert Schildt, “Java - The Complete Reference”, 11th edition, Oracle Press, 2019. ISBN: 978-1-26-044024-9.
2. Java tutorials, <https://www.w3schools.com/java/>.
3. Java tutorials, <https://www.javatpoint.com/>
4. Java tutorials, <https://www.thoughtco.com/>
5. Java tutorials, <https://www.guru99.com/>